

Artificial Intelligence

Prof. Chang-Hee Han

6

The universal workflow of machine learning

5.0 This chapter covers

▣ This chapter covers

- Steps for framing a machine learning problem
- Steps for developing a working model

6.1 Define the task

▣ 6.1.1 Frame the problem

- To define a machine learning problem, we usually need many detailed discussions with the client.
- The most important questions to ask first are:
  - ➢ What is the input data?
  - ➢ What do we want to predict?
  - ➢ We need training data to learn how to make predictions.
    - For example, if we want to classify the sentiment of movie reviews, the reviews must already have sentiment labels. With those labels, the model can learn how to classify new reviews.
- In this way, the availability of usable data often becomes a limiting factor at this stage.
- In many cases, we need to collect a new dataset and add labels ourselves.

6.1 Define the task

▣ 6.1.1 Frame the problem

- What Kind of Problem Is It?
  - ➢ Is it a binary classification problem?
  - ➢ A multi-class classification problem?
  - ➢ A scalar regression problem?
  - ➢ A vector regression problem?
  - ➢ A multi-label multi-class classification problem?
  - ➢ Or maybe image segmentation? Ranking?
  - ➢ Or is it a different type of problem like clustering, generation, or reinforcement learning?
- Sometimes, machine learning is not the best way to understand the data.
- In those cases, other methods such as statistical analysis (learned in school) may be better.

6.1 Define the task

▣ 6.1.1 Frame the problem

➢ Examples

- Image search engine project → Multi-label, multi-class classification task
- Spam detection project → Binary classification task
- If we define “offensive content” as a separate class → 3-class classification task
- Music recommendation engine → Better solved with matrix factorization (collaborative filtering) than deep learning
- Credit card fraud detection project → Binary classification task
- Click-through rate prediction project → Scalar regression task

6.1 Define the task

▣ 6.1.1 Frame the problem

- Problem Definition
  - ➢ What solutions already exist?
  - ➢ Your customers may already have algorithms made of many complicated if statements to handle spam filtering or detect fraud in credit card transactions.
  - ➢ Maybe there are people now who monitor these processes and handle them manually.
  - ➢ For example, monitoring a conveyor belt in a cookie factory and removing bad cookies manually, or making a music playlist for a user who likes a certain artist.
  - ➢ We need to understand with what kind of system, and how, these things are working.

6.1 Define the task

▣ 6.1.1 Frame the problem

- Problem Definition
  - ➢ Are there special restrictions to consider?
  - ➢ For example, an app that uses a spam detection system may be strongly encrypted end-to-end.
  - ➢ This means the spam detection model must be installed on the user's phone, and it can only be trained with external datasets.
  - ➢ A cookie filtering model may have time restrictions, so it has to run not on a remote server, but directly on embedded devices in the factory.
  - ➢ You must clearly understand the exact conditions that your work needs to satisfy.

6.1 Define the task

▣ 6.1.1 Frame the problem

- Problem Definition
  - ➢ After investigation, you will know what the input and target are, and in a broad sense, what kind of machine learning task matches this problem.
  - ➢ Remember the assumptions we make at this stage:
    - Assume that the given input can predict the target.
    - Assume that the available data (or data that will soon be collected) has enough information to learn the relationship between input and output.
  - ➢ These assumptions are uncertain until we actually get a working model.
  - ➢ We must wait and test to confirm.
  - ➢ Not all problems can be solved with machine learning.
  - ➢ Even if we collect samples of input X and target Y, there may not be enough information in X to predict Y.
  - ➢ For example, if we try to predict stock prices only based on recent market fluctuations, the chance of failure is high.
    - This is because past prices may not contain enough useful information for prediction.

6.1 Define the task

▣ 6.1.2 Collect a dataset

- After understanding the task and knowing the input and target, the next step is to collect data.
- In most machine learning projects, this step is the hardest, most time-consuming, and most expensive.
- Example: Image Search Engine Project
  - ➢ First, choose labels for classification.
  - ➢ For example, define 10,000 image categories.
  - ➢ Then, using these labels, users must manually tag the millions of uploaded images.

6.1 Define the task

▣ 6.1.2 Collect a dataset

- Example: Spam Detection in Chat Apps
  - ➢ Since user conversations are encrypted end-to-end, they cannot be used to train the model.
  - ➢ A separate dataset of unfiltered social media posts is needed, and these must be tagged as spam, offensive, or allowed.
- Example: Music Recommendation Engine
  - ➢ Can use user records such as “likes.”
  - ➢ No need to collect new data.
- Example: Click-through Prediction Project
  - ➢ Can use past advertising click-through data collected over years.

6.1 Define the task

▣ 6.1.2 Collect a dataset

- A good dataset is an asset that is worth managing and investing in.
- If you can spend 50 more hours on a project, the most effective way to use that time is to collect more data, rather than only trying to find small improvements in modeling.

6.1 Define the task

▣ 6.1.2 Collect a dataset

- Invest in Data Annotation Infrastructure
  - ➢ The data annotation process decides the quality of the target, and in the end, it decides the quality of the model.
  - ➢ We must carefully think about which method to use:
    - Should we annotate the data ourselves?
    - Should we use a crowdsourcing platform such as **Mechanical Turk** to collect labels?
    - Should we use the service of a professional data labeling company?

6.1 Define the task

▣ 6.1.2 Collect a dataset

- Invest in Data Annotation Infrastructure
  - ➢ Outsourcing can temporarily save time and cost, but it may reduce control.
  - ➢ Services like Mechanical Turk do not cost much and can easily scale up, but annotation quality may suffer.

6.1 Define the task

6.1.2 Collect a dataset

- Invest in Data Annotation Infrastructure
  - To choose the best option, consider the current constraints of the task.
  - Should the person assigning labels to the data be an expert in that field, or can anyone add labels?
    - For example, labeling cats vs. dogs in images can be done by anyone.
    - But labeling dog breeds requires expert knowledge.
  - Creating annotations for fracture CT scans requires a medical degree.
  - If data annotation needs expert knowledge, can we train people for it?
    - If not, can we hire experts in that field?
  - Do you understand the annotation work done by experts?
    - If not, you must treat the dataset like a black box, and you cannot manually do feature engineering.
  - This may not be fatal, but it can become a limitation.

6.1 Define the task

▣ 6.1.2 Collect a dataset

- Invest in Data Annotation Infrastructure
  - ➢ If you decide to create data labels internally, think carefully about what annotation software to use.
  - ➢ You may need to develop such software yourself.
  - ➢ Using productive data annotation software can save a lot of time, so it is worth investing in it at the early stage of the project.

6.1 Define the task

▣ 6.1.2 Collect a dataset

- Be careful about unrepresentative data
  - ➢ A machine learning model can only understand inputs that are similar to the samples it has seen before.
  - ➢ It is important that the training data represents the data found in the real product environment.
  - ➢ These considerations must be the foundation of all data collection work.

6.1 Define the task

▣ 6.1.2 Collect a dataset

- Be careful about unrepresentative data
  - ➢ Let's imagine we develop an app that finds the name of a dish from a food photo taken by the user.
  - ➢ We train the model using food photos from a popular image-sharing social network.
  - ➢ As deployment time gets closer, feedback from real users begins to arrive.
  - ➢ Users report that the app often fails to recognize the name of the dish, maybe 1 out of 10 times.
  - ➢ **What happened?**
  - ➢ In the test set, the accuracy was over 90%!
  - ➢ But when we checked the user-uploaded data, it turned out that:
    - Random smartphone photos taken in random restaurants
    - Are very different from the high-quality, well-lit, professional food photos used for training.
  - ➢ In other words, the training data did not represent the real product environment data.
  - ➢ A big mistake was made — welcome to the real world of machine learning.

6.1 Define the task

▣ 6.1.2 Collect a dataset

- Be careful about unrepresentative data
  - ➢ If possible, collect data directly from the environment where the model will be used.
  - ➢ For example, a movie review sentiment model should be applied to IMDB reviews, not Yelp restaurant reviews or Twitter messages.
  - ➢ If you want to evaluate tweet sentiment, collect and annotate real tweets from users similar to your target users.
  - ➢ If it is impossible to train with data collected from the product environment, you must fully understand the gap between training data and real data, and actively work to reduce this gap.

6.1 Define the task

6.1.2 Collect a dataset

- Concept Drift
  - Concept Drift = When data patterns change over time
  - Happens in almost all real-world problems
  - Especially critical with user-generated data
- Effects
  - Model accuracy decreases as data changes
  - Old training data → less useful today
  - Example: A model trained in 2013 music data may not work well now
  - Example: IMDB reviews from 2011 vs. reviews in 2020
- Causes
  - Language, expressions, genres, and user behavior change with time
  - Fraud detection is hardest → patterns change daily
- Solution
  - Continuous data collection, annotation, retraining

6.1 Define the task

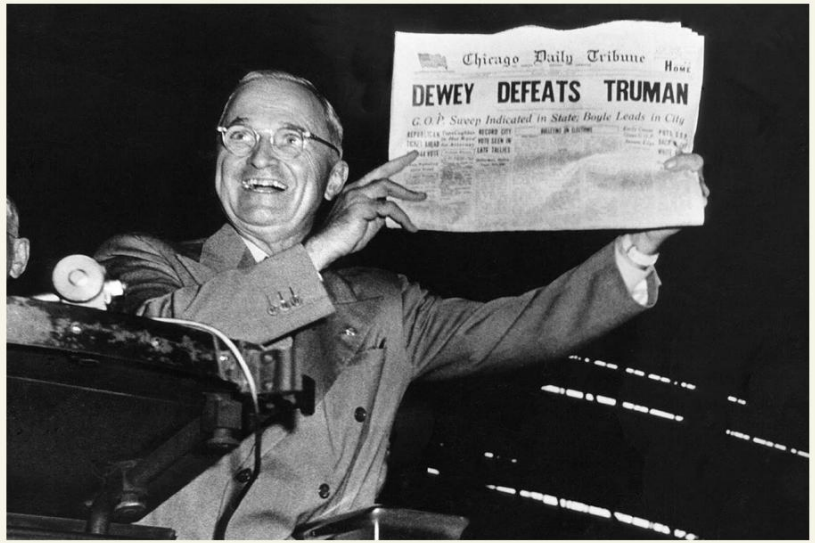
▣ 6.1.2 Collect a dataset

- Machine learning models only learn patterns from training data
- They can only recognize what they have seen before
- Using past data to predict the future assumes the future looks like the past
- In reality, this assumption often fails

6.1 Define the task

▣ 6.1.2 Collect a dataset

- Sampling Bias
  - ➢ Sampling Bias = Data collected does not represent the real population
  - ➢ Very common and dangerous mistake
- Famous Example (1948 US Presidential Election)
  - ➢ Chicago Tribune headline: *"Dewey Defeats Truman"*
  - ➢ Next morning, Truman actually won
  - ➢ Why? Telephone survey used → not representative
    - 1948 phone users = wealthier, conservative → more likely to vote Dewey
- Key Point
  - ➢ If training data is biased, predictions will also be biased



"DEWEY DEFEATS TRUMAN": A famous example of sampling bias

6.1 Define the task

6.1.3 Understand your data

- Treating a dataset like a black box is dangerous
- Before training, explore and visualize the data
- Goal: Understand predictive features and detect possible problems
- How to Explore Data
  - If images or text → check a few raw samples (and labels)
  - If numeric → plot histograms to see ranges and distributions
  - If location data → map it, look for clear patterns
  - If missing values → identify and handle them
- Key Point
  - Data exploration = foundation for feature engineering & reliable modeling

6.1 Define the task

▣ 6.1.3 Understand your data

- For classification tasks → check sample counts per class
  - ➢ Are the classes balanced?
  - ➢ If not, must handle imbalance
- Watch out for Target Leakage
  - ➢ Happens when training data contains information about the target that would not be available in real use
  - ➢ Example: Predicting cancer treatment using medical records that already say *"Patient diagnosed with cancer"* → invalid
  - ➢ Always check: Will these features exist in the product environment in the same form?
- Key Point
  - ➢ Ensure data is balanced and free from leakage to build a reliable model

6.1 Define the task

▣ 6.1.4 Choose a measure of success

- To control something, you must be able to measure it
- First, define what success means for the project
- Which metric matters most?
  - ➢ Accuracy? Precision? Recall? Customer retention?
- The chosen success metric will guide all technical decisions
- Must connect directly to the customer's business goals

6.1 Define the task

▣ 6.1.4 Choose a measure of success

- For balanced classification problems → common metrics:
  - **Accuracy**
  - **ROC AUC (Receiver Operating Characteristic Area Under Curve)**
- For imbalanced, ranking, or multi-label problems → use:
  - **Precision & Recall**
  - Or average of Accuracy / ROC AUC
- Defining a custom metric for your project is sometimes necessary
- To explore success metrics across many ML tasks → check Kaggle competitions
  - Kaggle shows diverse problems and evaluation metrics

6.2 Develop a model

▣ 6.2 Develop a model

- Once you know how to measure progress → you can start model development
- Most tutorials and research projects only focus on this step
- They assume:
  - Problem definition & data collection already done
  - Deployment & maintenance handled by others
- In reality → model development is just one step in the ML workflow, and not the hardest one
- The hardest parts are usually:
  - Problem definition
  - Data collection
  - Annotation
  - Cleaning

6.2 Develop a model

▣ 6.2.1 Prepare the data

- Deep learning models rarely use raw data directly
- Goal of preprocessing = make raw data easier for the model to use
- Common steps:
  - ➢ Vectorization
  - ➢ Normalization
  - ➢ Handling missing values
- Many preprocessing methods are domain-specific
  - ➢ Example: Text vs. Image data require different methods

6.2 Develop a model

▣ 6.2.1 Prepare the data

- **Vectorization**
  - ➢ In neural networks, all inputs & targets must be numeric tensors
    - (e.g., integers, floats; sometimes characters or text encoded as numbers)
  - ➢ Any data type (sound, image, text) must first be converted into a tensor
  - ➢ This step is called data vectorization
  - ➢ Examples
    - Text classification: convert sentences into a sequence of integers (word IDs)
    - Then apply one-hot encoding or convert to float32 tensors
    - Image classification or stock prediction: data is already in numeric vectors → skip this step

6.2 Develop a model

▣ 6.2.1 Prepare the data

- **Normalization**
  - ➢ Example: MNIST digit images
    - Raw data = grayscale integers (0–255)
    - Before feeding into the network:
      - ✓ Convert to float32
      - ✓ Divide by 255 → values scaled to range **0–1**
  - ➢ Example: Housing Price Prediction
    - Features (e.g., size, number of rooms, location) have very different ranges
    - Need to normalize features so they are comparable

6.2 Develop a model

▣ 6.2.1 Prepare the data

- **Normalization**
  - ➢ Typical Method
    - Scale each feature so that:
      - ✓ Mean = 0
      - ✓ Standard deviation = 1

6.2 Develop a model

▣ 6.2.1 Prepare the data

- Later, we will see a stricter way of normalization. It is often useful and used a lot, but not always necessary. (For example, it was not used in the number image classification example.)
  - ➢ Normalize so that the mean of each feature becomes 0.
  - ➢ Normalize so that the standard deviation of each feature becomes 1.
- Doing this with NumPy arrays is easy.

```
x -= x.mean(axis=0)
x /= x.std(axis=0)
```

Assuming x is a 2D data matrix of shape (samples, features)

6.2 Develop a model

▣ 6.2.1 Prepare the data

- **HANDLING MISSING VALUES**
  - ➢ Sometimes, some data values are missing.
  - ➢ For example, in a housing price prediction problem, the first feature (column with index 0 in the data) is the crime rate per person.
  - ➢ What if this feature is missing for some samples?
  - ➢ Missing values can appear in both the training data and the test data.

6.2 Develop a model

▣ 6.2.1 Prepare the data

- **HANDLING MISSING VALUES**
  - ➢ You can completely delete this feature, but it's not always necessary.
  - ➢ If the feature is categorical, it's safer to make a new category called "missing value."
    - The model will automatically learn what this means for the target.
  - ➢ If the feature is numerical, don't just fill it with a random value like "0."
    - This can create breaks (discontinuities) in the feature space and make it harder for the model to generalize.
    - Instead, consider filling the missing value with the mean or median of that feature.
  - ➢ You can also train another model to predict the missing values based on other features.

6.2 Develop a model

▣ 6.2.1 Prepare the data

- **HANDLING MISSING VALUES**
  - ➢ Suppose some categorical features might be missing in the test data.
  - ➢ If the network was trained only on data without missing values, it won't know how to handle missing ones!
  - ➢ In this case, you should intentionally create some training samples with missing values.
  - ➢ You can copy some training samples and remove the categorical features that might be missing in the test data.

6.2 Develop a model

▣ 6.2.2 Choose an evaluation protocol

- The goal of a model is to achieve generalization.
- Every modeling decision during development should be based on validation metrics that measure generalization performance.
- The goal of validation is to accurately estimate how well the model will perform in the real-world product environment, using success metrics (e.g., accuracy).
- The reliability of this process is very important for building a useful model.

6.2 Develop a model

▣ 6.2.2 Choose an evaluation protocol

- In Chapter 5, we learned about three evaluation methods:
  - **Hold-out validation:** Use when there is plenty of data.
  - **K-fold cross-validation:** A good choice when the number of samples is too small for hold-out validation.
  - **Repeated K-fold cross-validation:** Use when data is limited and very accurate model evaluation is needed.
- You can choose one of these methods.
- In most cases, the first one (hold-out) is enough.
- Always make sure the validation set is representative of the data.
- Also, make sure there are no overlapping samples between the training set and validation set.

6.2 Develop a model

▣ 6.2.3 Beat a baseline

- When starting to build models, the first goal is to achieve statistical power.
- In other words, create a small model that can perform better than a simple baseline.
- At this stage, focus on three key areas:
  - ➢ Feature engineering — Remove unhelpful features (feature selection) and use domain knowledge to create new, useful features.
  - ➢ Choosing the right structure — Decide what kind of model architecture to use:
    - Fully connected neural network, convolutional network, recurrent network, or transformer?
    - Is deep learning a good approach for this task, or should you use something else?
  - ➢ Selecting good training options —
    - What **loss function** should you use?
    - What **batch size** and **learning rate** should you choose?

6.2 Develop a model

▣ 6.2.3 Beat a baseline

- Picking the right loss function
  - ➢ In many cases, it's not possible to directly optimize the success metric of the problem.
  - ➢ Sometimes, there is no easy way to turn that metric into a loss function.
  - ➢ Most importantly, a loss function must be computable from each mini-batch of data (ideally even from a single data point) and differentiable (otherwise, backpropagation can't be used to train the network).
  - ➢ For example, the widely used classification metric ROC AUC cannot be directly optimized.
  - ➢ For classification tasks, we usually optimize another metric such as cross-entropy loss instead of ROC AUC.
  - ➢ Generally, as cross-entropy loss decreases, we can expect ROC AUC to increase.

6.2 Develop a model

▣ 6.2.3 Beat a baseline

- Picking the right loss function
  - ➢ The following table summarizes **which activation function and loss function** to choose for the **last layer**, depending on the **type of problem**.

Problem type	Last-layer activation	Loss function
Binary classification	`sigmoid`	`binary_crossentropy`
Multiclass, single-label classification	`softmax`	`categorical_crossentropy`
Multiclass, multilabel classification	`sigmoid`	`binary_crossentropy`

6.2 Develop a model

▣ 6.2.3 Beat a baseline

- In most cases, there are existing templates you can use when starting out.
- You are probably not the first person to build a spam detection system, a music recommendation engine, or an image classifier.
- Study prior work to find feature engineering methods and model structures that fit your task.
- Achieving statistical power is not always possible.
- If you try several reasonable network architectures and still can't beat a simple baseline, it may be a sign that your input data doesn't contain enough information.

6.2 Develop a model

▣ 6.2.3 Beat a baseline

- Remember these two assumptions:
  - ➢ You assume that the target can be predicted from the given inputs.
  - ➢ You assume that the available data (or the data you plan to collect) contains enough information to learn the relationship between inputs and outputs.
- These assumptions might be wrong.
- If that happens, you may need to go back to the beginning and rethink your approach.

6.2 Develop a model

▣ 6.2.4 Scale up: Develop a model that overfits

- Once you have a model with statistical power, ask whether it now performs well enough.
- Does the model have enough layers and parameters to properly represent the given problem?
- For example, a logistic regression model on the MNIST dataset may have statistical power but still not be powerful enough to solve the task well.
- Remember that machine learning is a balance between optimization and generalization.
- **The best model lies between underfitting and overfitting, that is, at the boundary between under-capacity and over-capacity.**
- To find where this boundary is, you often need to go past it first and then step back.

6.2 Develop a model

▣ 6.2.4 Scale up: Develop a model that overfits

- To find out how large your model should be, you first need to build an overfitted model.
- As we learned in Chapter 5, this is very easy to do:
  - ➢ Add more layers
  - ➢ Increase the size of the layers
  - ➢ Train for more epochs
- Always monitor both training and validation losses, along with the main training and validation metrics.
- When the validation performance starts to drop while the training performance keeps improving, your model has reached overfitting.

6.2 Develop a model

▣ 6.2.5 Regularize and tune your model

- If your model has achieved statistical power and can overfit, you're heading in the right direction.
- The goal now is to maximize generalization performance.
- This stage usually takes most of the time in model development.
- You repeatedly modify, train, and evaluate the model using validation data (not the test data).
- Keep adjusting and retraining until you get the best possible model.

6.2 Develop a model

▣ 6.2.5 Regularize and tune your model

- **You should try the following:**
  - ➢ Try a different model structure.
  - ➢ Add or remove layers.
  - ➢ Add dropout.
  - ➢ If the model is small, add L1 or L2 regularization.
  - ➢ Tune hyperparameters to find the best setup (e.g., number of units per layer, optimizer learning rate).
  - ➢ Optionally, try data curation or feature engineering:
    - Collect more data, add better annotations, or create improved features.
    - Remove features that seem **unhelpful**.

6.2 Develop a model

▣ 6.2.5 Regularize and tune your model

- You can use automated hyperparameter tuning tools like KerasTuner to automate much of this process.
- However, every time you tune the model using feedback from the validation process, you're leaking information from the validation data into the model.
- Repeating this a few times is usually not a big problem.
- But if you do it too many times, the model will eventually overfit to the validation process (even though it's never trained directly on the validation data).
- This reduces the reliability of the validation results.

6.2 Develop a model

▣ 6.2.5 Regularize and tune your model

- Once you've found a satisfactory model configuration, train the final model using all available data (training + validation data) for deployment.
- Then, evaluate it once on the test set.
- If the test performance is much worse than what you saw on the validation data, it means either:
  - ➢ The validation process was unreliable, or
  - ➢ The model overfitted to the validation data during hyperparameter tuning.
- In that case, it's better to use a more reliable evaluation method, such as repeated K-fold cross-validation.



Q & A

Prof. Chang-Hee Han